

6 Event-driven programming

In object-oriented programming, we model entities in the world as objects. A natural question is how do we model interaction between these objects. One way of modelling this behaviour is interpreting a method call `x.DoIt()` as passing object `x` the message `DoIt`. Another way is for objects to register interest in and react to events that other objects raise.

For example, consider a `Button` object on a `Window`. One event of interest is when the button is pressed. When this happens, the program may want to execute some logic such as closing the window. To do this, the `Button` object makes available the event, call it `OnPress`. The program then registers for that event with the `Button`. Now, when the `Button` is clicked, the `Button` notifies all objects that registered for the `OnPress` event.

This is commonly called the observer pattern: an `Observable` object is one whom has an event of interest to an `Observer` object. Java implements event-driven programming with these two interfaces.

6.1 Delegates

C#, on the other hand, provides explicit language support for events. The infrastructure begins with delegates. Delegates are a type that defines a method signature. By defining such a type, we can now pass methods around as objects.

```
public delegate int BinOp(int x, int y);
public static class BinOpTest
{
    public static int Add(int x, int y) { return x + y; }
    public static int Sub(int x, int y) { return x - y; }

    public static void Test()
    {
```

```
        // Note either method of creating a BinOp works.
```

The second method

```
        // is a more recent version called method group
```

conversion.

```
        BinOp op1 = new BinOp(Add);
        BinOp op2 = Sub;
        Console.WriteLine(op1(1, 1));
```